Algoritmos de Programação Dinâmica

Prof. Ricardo P. Mesquita

O que é Programação Dinâmica?

- Ferramenta poderosa para resolver problemas de *otimização*
- Quando usamos?
  - Quando queremos encontrar a *melhor* (ou *pior*) solução para ***todas*** as instâncias de um determinado problema
  - Quando ***não é fácil*** chegar a uma sequência ótima de decisões ***sem*** testar todas as sequências possíveis para então escolher a melhor
  - Quando os subproblemas não são independentes, isto é, quando subproblemas compartilham sub-subproblemas
  - Quando o algoritmo direto tem complexidade exponencial,
    - o algoritmo desenvolvido por programação dinâmica é **polinomial**, reduzindo drasticamente o tempo de execução.
    - Outras vezes, a complexidade continua exponencial, mas de ordem mais baixa.

O que é Programação Dinâmica?

- A ***cada passo*** do algoritmo são eliminadas sub-soluções que certamente não farão parte da solução ótima
- Reduz drasticamente o número total de sequências viáveis através de um mecanismo que evita aquelas que sabidamente não podem resultar ótimas
- Resolve problemas combinando soluções de subproblemas
- Os algoritmos de programação dinâmica resolvem cada subproblema apenas uma vez, gravando as respostas em uma tabela, evitando assim o trabalho de recalcular a resposta toda vez que o subproblema é encontrado

Etapas Necessárias

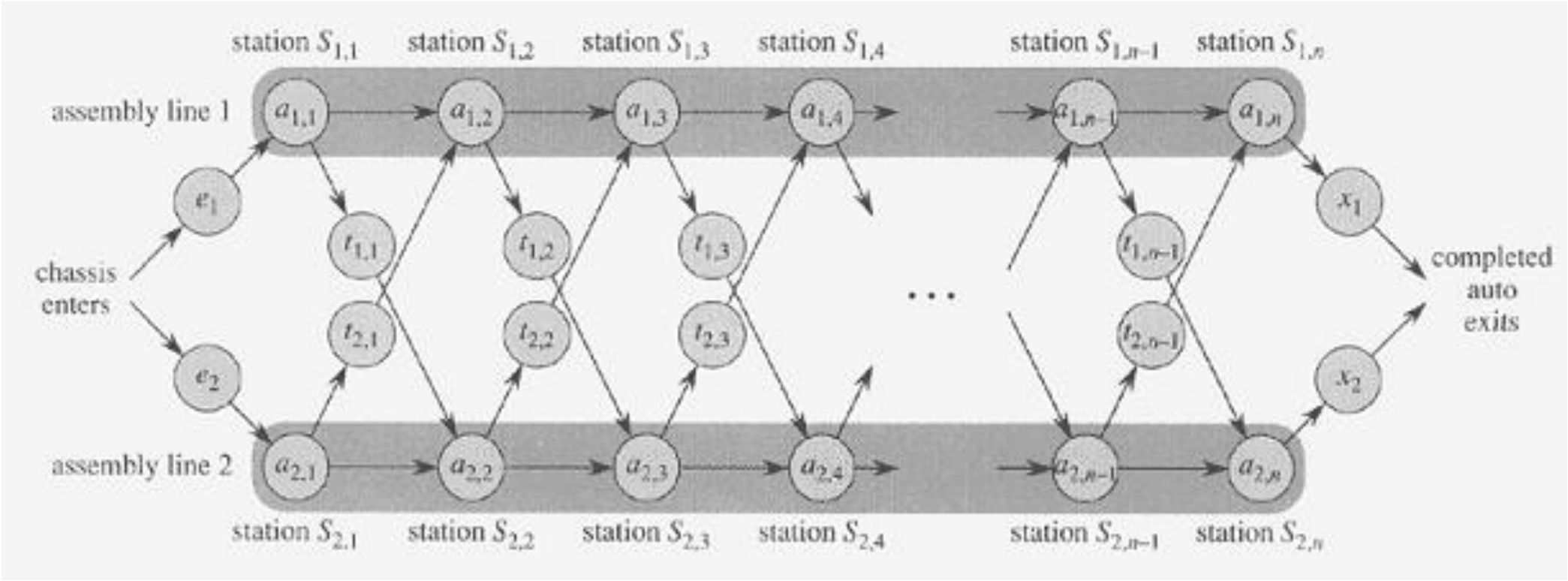
- Caracterizar a estrutura de uma solução ótima
- Definir recursivamente o valor de uma solução ótima
- Calcular o valor de **uma** solução ótima em um processo ***bottom-up***
- Construir uma solução ótima a partir de informações calculadas
- Vamos ver alguns exemplos...

O Problema da Linha de Montagem

- Há duas linhas de montagem, cada uma com n estações
- A j-ésima estação na linha i é denotada por $S_{i,j}$ e o tempo de montagem nessa estação é $a_{i,j}$
- Um chassi entra na fábrica e vai para a linha i (onde $i = 1$ ou 2), demorando o tempo e_i.
- Depois de passar pela j-ésima estação em uma linha, o chassi vai para a ($j + 1$)-ésima estação de uma das linhas
- Não há custo de transferência se ele ficar na mesma linha, mas demora o tempo $t_{i,j}$ para transferir a peça para a outra linha após a estação $S_{i,j}$
- Depois de sair da n-ésima estação em uma linha, decorre o tempo x_i para a peça estar pronta

O Problema da Linha de Montagem

- **Problema**: determinar que estações escolher na linha 1 e quais escolher na linha 2 para minimizar o tempo total de passagem de uma peça pela fábrica



O Problema da Linha de Montagem

Por que é um problema difícil?

- Se recebemos uma lista de quais estações usar na linha 1 e quais estações usar na linha 2 calculamos no tempo $\Theta(n)$, mas...
- Existem 2^n maneiras possíveis de se escolher as estações...
- Assim, enumerar todos os caminhos, calcular o tempo de cada um e escolher o melhor exigiria o tempo $\Omega(2^n)$ – inviável para n grande!

O Problema da Linha de Montagem

Qual a estrutura do melhor caminho?

- Analisando o problema percebemos que uma solução ótima para o problema contém em seu interior soluções ótimas para cada subproblema...
- Chamamos essa propriedade de ***subestrutura ótima***
- Se examinarmos um caminho mais rápido através da estação $S_{1,j}$, ele tem de passar pela estação $j - 1$ na linha 1 **ou** na linha 2
- Desse modo, o caminho mais rápido pela estação $S_{1,j}$ é:
  - O caminho mais rápido pela estação $S_{1,j-1}$, e depois *diretamente* pela estação $S_{1,j}$.
  - O caminho mais rápido pela estação $S_{2,j-1}$, *uma transferência* da linha 2 para a linha 1, e depois através da estação $S_{1,j}$.

O Problema da Linha de Montagem

Qual a estrutura do melhor caminho?

- Simetricamente, o caminho mais rápido através da estação $S_{2,j}$ é:
  - O caminho mais rápido pela estação $S_{2,j-1}$, e depois *diretamente* pela estação $S_{2,j}$.
  - O caminho mais rápido pela estação $S_{1,j-1}$, *uma transferência* da linha 1 para a linha 2, e depois através da estação $S_{2,j}$.
- **Ou seja**:

  Para resolver o problema, precisamos resolver os subproblemas de encontrar caminhos mais curtos pela estação $j-1$ em ambas as linhas
- **Desse modo**:

  Podemos elaborar uma solução ótima para uma instância do problema de programação dinâmica da linha de montagem através da elaboração de soluções ótimas para os subproblemas

O Problema da Linha de Montagem

Qual é a solução recursiva?

- Para nos situarmos...

  Precisamos definir recursivamente o valor de uma solução ótima em termos das soluções ótimas para subproblemas

- **Subproblema**:

  Encontrar o caminho mais rápido pela estação j em ambas as linhas

- Formulando:
  - $f_i[\, j \,]$ = melhor tempo do ponto de partida até $S_{i,\, i}$
  - f^* = melhor tempo em todo o percurso

$$f^* = \min(f_1[\, n \,] + x_1, f_2[\, n \,] + x_2)$$

O Problema da Linha de Montagem

Qual é a solução recursiva?

- Para chegar à estação 1 em qualquer linha, basta um chassi ir diretamente até essa estação:
  - $f_1[1] = e_1 + a_{1,1}$
  - $f_2[1] = e_2 + a_{2,1}$
- Lembrando:

  O melhor caminho pela estação $S_{1,j}$ é o melhor caminho pela estação $S_{1,j-1}$, e depois diretamente pela estação $S_{1,j}$, ou o melhor caminho pela estação $S_{2,j-1}$, uma transferência da linha 2 para a linha 1, e depois através da estação $S_{1,j}$.
- Assim:
  - $f_1[j] = f_1[j-1] + a_{1,j}$ (primeiro caso)
  - $f_1[j] = f_2[j-1] + t_{2,j-1} + a_{1,j}$

O Problema da Linha de Montagem

Qual é a solução recursiva?

- Desse modo:

  $f_1[j] = \min(f_1[j-1] + a_{1,j}, f_2[j-1] + t_{2,j-1} + a_{1,j})$

  para $j = 2, 3, \ldots, n$

- Simetricamente:

  $f_2[j] = \min(f_2[j-1] + a_{2,j}, f_1[j-1] + t_{1,j-1} + a_{2,j})$

  para $j = 2, 3, \ldots, n$

- Combinando:

$$f_1[j] = \begin{cases} e_1 + a_{1,1}, \; se \; j = 1, \\ \min(f_1[j-1] + a_{1,j}, \quad f_2[j-1] + t_{2,j-1} + a_{1,j}), \; se \; j \geq 2. \end{cases}$$

$$f_2[j] = \begin{cases} e_2 + a_{2,1} \; se \; j = 1 \\ \min(f_2[j-1] + a_{2,j}, \quad f_1[j-1] + t_{1,j-1} + a_{2,j}) \; se \; j \geq 2 \end{cases}$$

O Problema da Linha de Montagem

Como calcular os melhores tempos?

- Escrever um algoritmo recursivo baseado nas equações anteriores ***não é*** a melhor solução!
- Note:

Seja $r_i(j)$ o número de referências feitas a $f_i[j]$ no algoritmo recursivo. Pelas recursões dadas, temos que:

$$r_1(j) = r_2(j) = r_1(j + 1) + r_2(j + 1)$$

donde

$$r_i(j) = 2^{n-j} .$$

Ou seja: $f_1[1]$, sozinho, é referenciado 2^{n-1} vezes!!!

O Problema da Linha de Montagem

Como calcular os melhores tempos?

- Podemos melhorar isso se calcularmos os valores $f_i[j]$ em uma ordem diferente...
- Para $j \geq 2$, cada valor de $f_i[j]$ depende apenas dos valores de $f_1[j-1]$ e $f_2[j-1]$
- Calculamos os valores $f_i[j]$ na ordem *crescente* dos números de estação j
- Dessa forma, teremos um tempo $\Theta(n)$

O Problema da Linha de Montagem

Como calcular os melhores tempos?

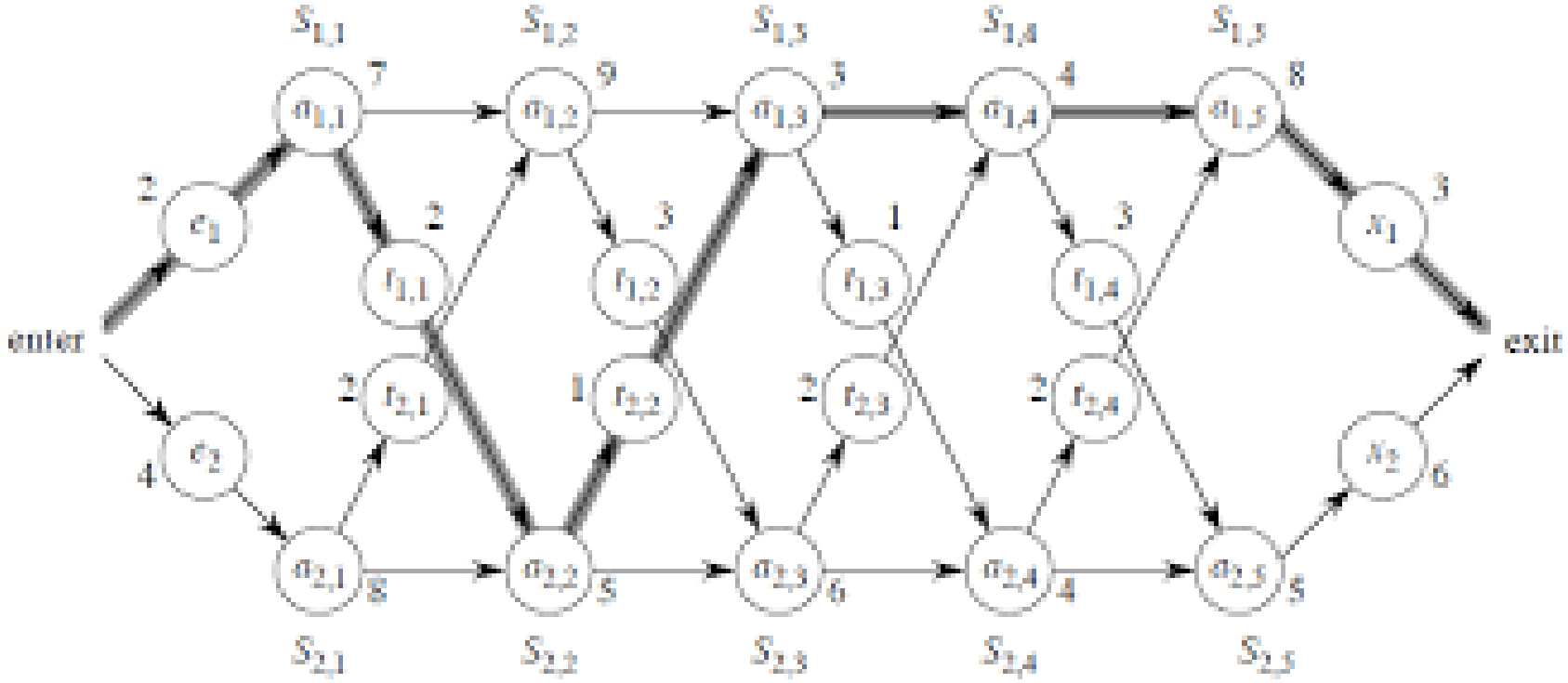
- Fica algo assim:

$f_1[1] \leftarrow e_1 + a_{1,1}$
$f_2[1] \leftarrow e_2 + a_{2,1}$
for $j \leftarrow 2$ **to** n **do**
    if $f_1[j-1] + a_{1,j} \leq f_2[j-1] + t_{2,j-1} + a_{1,j}$ **then**
        $f_1[j] \leftarrow f_1[j-1] + a_{1,j}$
        $l_1[j] \leftarrow 1$
    else
        $f_1[j] \leftarrow f_2[j-1] + t_{2,j-1} + a_{1,j}$
        $l_1[j] \leftarrow 2$
    if $f_2[j-1] + a_{2,j} \leq f_1[j-1] + t_{1,j-1} + a_{2,j}$ **then**
        $f_2[j] \leftarrow f_2[j-1] + a_{2,j}$
        $l_2[j] \leftarrow 2$
    else
        $f_2[j] \leftarrow f_1[j-1] + t_{1,j-1} + a_{2,j}$
        $l_2[j] \leftarrow 1$
if $f_1[n] + x_1 \leq f_2[n] + x_2$ **then**
    $f^* \leftarrow f1[n] + x_1$
    $l^* \leftarrow 1$
else $f^* \leftarrow f_2[n] + x_2$
    $l^* \leftarrow 2$

O Problema da Linha de Montagem

O melhor caminho?

- Tendo calculado os valores dos parâmetros e armazenando-os em vetores, basta, agora, percorrer esses vetores apresentando o caminho...



Sintetizando...

Como caracterizar uma subestrutura ótima?

1. Mostrar que uma solução para o problema consiste em *fazer uma escolha*; essa escolhe deixa *um ou mais* subproblemas a serem resolvidos
2. Supor que, para um dado problema, você tenha a solução ótima. (Não se preocupe aqui em *como* determinar essa escolha.)
3. Dada a escolha (do item 2), determinar quais subproblemas resultam dela e como caracterizar melhor o espaço de subproblemas resultante
4. Mostrar que as soluções para os subproblemas usados dentro da solução ótima para o problema devem elas próprias ser ótimas

Sintetizando...

Como as subestruturas ótimas variam no domínio do problema?

- No número de subproblemas que são usados em uma solução ótima para o problema original
- No número de opções que temos na determinação de qual(is) subproblema(s) usar em uma solução ótima

Quando temos superposição de problemas?

- Quando um algoritmo recursivo reexamina o mesmo problema inúmeras vezes!
  - Os algoritmos de programação dinâmica tiram proveito de subproblemas superpostos resolvendo cada subproblema uma vez e depois armazenando a solução em uma tabela, onde ela possa ser examinada quando necessário

O Problema da Maior Subsequência Comum

- Considere o problema de comparar o DNA de dois (ou mais) organismos diferentes
- Uma *subsequência* de uma determinada sequência é apenas a sequência dada com 0 ou mais elementos omitidos
- *Em termos formais*:

  Dada uma sequência $X = \langle x_1, x_2, \ldots, x_n \rangle$, outra sequência $Z = \langle z_1, z_2, \ldots, z_n \rangle$ é uma *subsequência* de X se existe uma sequência estritamente crescente $\langle i_1, i_2, \ldots, i_k \rangle$ de índices de X tais que, para todo $j = 1, 2, \ldots, k$, temos $x_{ij} = z_j$.
- Dadas duas sequências X e Y, dizemos que uma sequência Z é uma *subsequência comum* de X e Y se Z é uma subsequência de X e Y ao mesmo tempo.
- No problema proposto, *dadas duas sequências, desejamos encontrar uma subsequência comum de comprimento máximo*.

O Problema da Maior Subsequência Comum

- Exemplo:

  X = {A B C B D A B }, Y= {B D C A B A}

  A *maior subsequencia comum* é:

  X = A **B** **C** **B** D **A** B

  Y = **B** D **C** A **B** **A**

O Problema da Maior Subsequência Comum

Etapa 1: *Caracterização de uma subsequência comum mais longa*

- Solução por força bruta: comparar cada subsequencia de X com os símbolos de Y.
- Se $|X| = m$, $|Y| = n$: 2^m subsequências de X
- Solução por força bruta é $O(n\ 2^m)$ $\therefore$ *impraticável* para longas sequências!
- Mas: Note que o LCS exibe *subestrutura ótima*: soluções de subproblemas fazem parte da solução final!
- Considere a seguinte definição:
  - Dada uma sequência $X = \langle x_1, x_2, \ldots, x_m \rangle$, definimos o i-ésimo ***prefixo*** de X, para $i = 0, 1, \ldots, m$, como $X_i = \langle x_1, x_2, \ldots, x_i \rangle$.
- As classes naturais de subproblemas correspondem a pares de prefixos das duas sequências

O Problema da Maior Subsequência Comum

Etapa 1: *Caracterização de uma subsequência comum mais longa*

- **Teorema:** Sejam as sequências $X = \langle x_1, x_2, \ldots, x_m \rangle$, $Y = \langle y_1, y_2, \ldots, y_n \rangle$, e seja $Z = \langle z_1, z_2, \ldots, z_l \rangle$ qualquer LCS de X e Y. Então:
  - Se $x_m = y_n$, então $z_k = x_m = y_n$, e Z_{k-1} é uma LCS de X_{m-1} e Y_{n-1}.
  - Se $x_m \neq y_n$, então se $z_k \neq x_m$ então Z é uma LCS de X_{m-1} e Y.
  - Se $x_m \neq y_n$, então se $z_k \neq y_m$ então Z é uma LCS de X e Y_{n-1}.
- A caracterização do teorema mostra que uma LCS de duas sequências contém dentro dela uma LCS de prefixos das duas sequências.
- Desse modo, o problema de LCS tem uma propriedade de subestrutura ótima!

O Problema da Maior Subsequência Comum

Etapa 2: *Uma solução recursiva*

- Achar LCS para prefixos de X e Y
  - Sejam X_i, Y_j prefixos de *X* e *Y* de tamanhos *i* e *j* respectivamente
- $c[i, j]$ é o tamanho da LCS de X_i e Y_j
- Logo, LCS de *X* e *Y* vai ser guardado em $c[m, n]$

$$c[i,j] = \begin{cases} c[i-1, j-1]+1 & \text{if } x[i] = y[j], \\ \max(c[i, j-1], c[i-1, j]) & \text{otherwise} \end{cases}$$

- Caso base: $i = j = 0$ (substrings vazias de *X* e *Y*)
- Se X_0 e/ou Y_0 são strings vazias: para todo *i* e *j*: $c[0, j] = c[i, 0] = 0$

O Problema da Maior Subsequência Comum

Etapa 2: *Uma solução recursiva*

$$c[i,j] = \begin{cases} c[i-1, j-1]+1 & \text{if } x[i] = y[j], \\ \max(c[i, j-1], c[i-1, j]) & \text{otherwise} \end{cases}$$

- **Primeiro caso:**
  - $x[i] = y[j]$: mais um símbolo em X e Y confere
  - Logo, LCS para X_i e Y_j é igual ao LCS de X_{i-1} e Y_{i-1} mais 1
- **Segundo caso:**
  - $x[i] \neq y[j]$
  - Se os símbolos não casam, não podemos melhorar a nossa resposta: (i.e.: $\max(LCS(X_i, Y_{j-1})$ e $LCS(X_{j-1}, Y_j))$

O Problema da Maior Subsequência Comum

Etapa 3: *Como calcular o comprimento de uma LCS?*

- Com base nas equações vistas, poderíamos escrever facilmente um algoritmo recursivo de tempo exponencial para calcular o comprimento de uma LCS de duas seqüências
- Contudo, como existem "apenas" $\Theta(mn)$ subproblemas distintos, podemos usar a PD para calcular as soluções de modo bottom-up
- O código a seguir toma duas seqüências $X = \langle x_1, x_2, \ldots, x_m \rangle$ e $Y = \langle y_1, y_2, \ldots, y_n \rangle$, como entrada
- Armazena os valores $c[i, j]$ em uma tabela $c[0..m, 0..n]$ cujas entradas são calculadas em ordem de linha principal

O Problema da Maior Subsequência Comum

- Fica algo assim:

```
LCS(X, Y)
m = length(X)
n  = length(Y)
for i = 1 to m    c[i, 0] = 0    // caso base: Y0
    for j = 1 to n        c[0, j] = 0    // caso base: X0
for i = 1 to m
    for j = 1 to n
        if ( xi == yj )
            c[i, j] = c[i - 1, j - 1] + 1
        else c[i, j] = max( c[i - 1, j], c[i, j - 1] )
return c
```

O Problema da Maior Subsequência Comum

Exemplo:

- Considere:
  - X = ABCB
  - Y = BDCAB
- Determine o tamanho da LCS de X e Y.

O Problema da Maior Subsequência Comum

i	j	0	1	2	3	4	5
		Yj	**B**	**D**	**C**	**A**	**B**
0	Xi						
1	**A**						
2	**B**						
3	**C**						
4	**B**						

X = ABCB; m = |X| = 4
Y = BDCAB; n = |Y| = 5

ABCB
BDCAB

O Problema da Maior Subsequência Comum

i	j	0	1	2	3	4	5
		Yj	**B**	**D**	**C**	**A**	**B**
0	Xi	**0**	**0**	**0**	**0**	**0**	**0**
1	**A**	**0**					
2	**B**	**0**					
3	**C**	**0**					
4	**B**	**0**					

for i = 1 to m
for j = 1 to n

c[i,0] = 0
c[0,j] = 0

ABCB
BDCAB

O Problema da Maior Subsequência Comum

i	j	0	1	2	3	4	5
		Yj	B	D	C	A	B
0	Xi	0	0	0	0	0	0
1	A	0	0				
2	B	0					
3	C	0					
4	B	0					

if (x_i == y_j)

$c[i, j] = c[i - 1, j - 1] + 1$

else $c[i, j] = \max(c[i - 1, j], c[i, j - 1])$

ABCB

BDCAB

O Problema da Maior Subsequência Comum

i	j	0	1	2	3	4	5
		Yj	**B**	**D**	**C**	**A**	**B**
0	Xi	**0**	**0**	**0**	**0**	**0**	**0**
1	**A**	**0**	**0**	**0**	**0**		
2	**B**	**0**					
3	**C**	**0**					
4	**B**	**0**					

if (x_i == y_j)

$c[i, j] = c[i - 1, j - 1] + 1$

else $c[i, j] = \max(c[i - 1, j], c[i, j - 1])$

ABCB

BDCAB

O Problema da Maior Subsequência Comum

i	j	0	1	2	3	4	5
		Yj	**B**	**D**	**C**	**A**	**B**
0	Xi	**0**	**0**	**0**	**0**	**0**	**0**
1	**A**	**0**	**0**	**0**	**0**	**1**	
2	**B**	**0**					
3	**C**	**0**					
4	**B**	**0**					

if ($x_i == x_j$)

$c[i, j] = c[i - 1, j - 1] + 1$

else $c[i, j] = \max(c[i - 1, j], c[i, j - 1])$

ABCB

BDCAB

O Problema da Maior Subsequência Comum

i	j	0	1	2	3	4	5
		Yj	**B**	**D**	**C**	**A**	**B**
0	Xi	**0**	**0**	**0**	**0**	**0**	**0**
1	**A**	**0**	**0**	**0**	**0**	**1** →	**1**
2	**B**	**0**					
3	**C**	**0**					
4	**B**	**0**					

if ($x_i == y_j$)

$c[i, j] = c[i - 1, j - 1] + 1$

else $c[i, j] = \max(c[i - 1, j], c[i, j - 1])$

ABCB

BDCAB

O Problema da Maior Subsequência Comum

i	j	0	1	2	3	4	5
		Yj	**B**	**D**	**C**	**A**	**B**
0	Xi	**0**	**0**	**0**	**0**	**0**	**0**
1	**A**	**0**	**0**	**0**	**0**	**1**	**1**
2	**B**	**0**	**1**				
3	**C**	**0**					
4	**B**	**0**					

if (x_i == y_j)

c[i, j] = c[i - 1, j - 1] + 1

else c[i, j] = max(c[i - 1, j], c[i, j - 1])

ABCB

BDCAB

O Problema da Maior Subsequência Comum

i	j	0	1	2	3	4	5
		Yj	B	D	C	A	B
0	Xi	0	0	0	0	0	0
1	A	0	0	0	0	1	1
2	B	0	1	1	1	1	
3	C	0					
4	B	0					

if ($x_i == y_j$)

$c[i, j] = c[i - 1, j - 1] + 1$

else $c[i, j] = \max(c[i - 1, j], c[i, j - 1])$

ABCB

BDCAB

O Problema da Maior Subsequência Comum

i	j	0	1	2	3	4	5
		Yj	**B**	**D**	**C**	**A**	**B**
0	Xi	**0**	**0**	**0**	**0**	**0**	**0**
1	**A**	**0**	**0**	**0**	**0**	**1**	**1**
2	**B**	**0**	**1**	**1**	**1**	**1**	**2**
3	**C**	**0**					
4	**B**	**0**					

if (x_i == y_j)

c[i, j] = c[i - 1, j - 1] + 1

else c[i, j] = max(c[i - 1, j], c[i, j - 1])

ABCB

BDCAB

O Problema da Maior Subsequência Comum

i	j	0	1	2	3	4	5
		Yj	B	D	C	A	B
0	Xi	0	0	0	0	0	0
1	A	0	0	0	0	1	1
2	B	0	1	1	1	1	2
3	C	0	1	1			
4	B	0					

if ($x_i == y_j$)

$c[i, j] = c[i - 1, j - 1] + 1$

else $c[i, j] = \max(c[i - 1, j], c[i, j - 1])$

ABCB

BDCAB

O Problema da Maior Subsequência Comum

i	j	0	1	2	3	4	5
		Yj	**B**	**D**	**C**	**A**	**B**
0	Xi	**0**	**0**	**0**	**0**	**0**	**0**
1	**A**	**0**	**0**	**0**	**0**	**1**	**1**
2	**B**	**0**	**1**	**1**	**1**	**1**	**2**
3	**C**	**0**	**1**	**1**	**2**		
4	**B**	**0**					

if (x_i == y_j)

c[i, j] = c[i - 1, j - 1] + 1

else c[i, j] = max(c[i - 1, j], c[i, j - 1])

ABCB

BDCAB

O Problema da Maior Subsequência Comum

i	j	0	1	2	3	4	5
		Yj	**B**	**D**	**C**	**A**	**B**
0	Xi	**0**	**0**	**0**	**0**	**0**	**0**
1	**A**	**0**	**0**	**0**	**0**	**1**	**1**
2	**B**	**0**	**1**	**1**	**1**	**1**	**2**
3	**C**	**0**	**1**	**1**	**2**	**2**	**2**
4	**B**	**0**					

if ($x_i == y_j$)

$c[i, j] = c[i - 1, j - 1] + 1$

else $c[i, j] = \max(c[i - 1, j], c[i, j - 1])$

ABCB

BDCAB

O Problema da Maior Subsequência Comum

i	j	0	1	2	3	4	5
		Yj	B	D	C	A	B
0	Xi	0	0	0	0	0	0
1	A	0	0	0	0	1	1
2	B	0	1	1	1	1	2
3	C	0	1	1	2	2	2
4	B	0	1				

if ($x_i == y_j$)

$c[i, j] = c[i - 1, j - 1] + 1$

else $c[i, j] = \max(c[i - 1, j], c[i, j - 1])$

ABCB

BDCAB

O Problema da Maior Subsequência Comum

i	j	0	1	2	3	4	5
		Yj	B	D	C	A	B
0	Xi	0	0	0	0	0	0
1	A	0	0	0	0	1	1
2	B	0	1	1	1	1	2
3	C	0	1	1	2	2	2
4	B	0	1	1	2	2	

if ($x_i == y_j$)

$c[i, j] = c[i - 1, j - 1] + 1$

else $c[i, j] = \max(c[i - 1, j], c[i, j - 1])$

ABCB

BDCAB

O Problema da Maior Subsequência Comum

i	j	0	1	2	3	4	5
		Yj	**B**	**D**	**C**	**A**	**B**
0	Xi	**0**	**0**	**0**	**0**	**0**	**0**
1	**A**	**0**	**0**	**0**	**0**	**1**	**1**
2	**B**	**0**	**1**	**1**	**1**	**1**	**2**
3	**C**	**0**	**1**	**1**	**2**	**2**	**2**
4	**B**	**0**	**1**	**1**	**2**	**2**	**3**

if ($x_i == y_j$)

$c[i, j] = c[i - 1, j - 1] + 1$

else $c[i, j] = \max(c[i - 1, j], c[i, j - 1])$

ABCB

BDCAB

O Problema da Maior Subsequência Comum

Etapa 4: *A construção de uma LCS*

- Cada $c[i, j]$ depende de $c[i-1, j]$, $c[i, j-1]$ e $c[i-1, j-1]$
- Podemos identificar como cada $c[i, j]$ foi obtido:

2	2
2	3

Por exemplo,
$c[i, j] = c[i - 1, j - 1] + 1 = 2 + 1 = 3$

- Sabemos que:

$$c[i,j] = \begin{cases} c[i-1, j-1]+1 & \text{if } x[i] = y[j], \\ \max(c[i, j-1], c[i-1, j]) & \text{otherwise} \end{cases}$$

O Problema da Maior Subsequência Comum

- Começar de $c[m, n]$ e retornar
- Sempre que $c[i - 1, j - 1] = c[i, j - 1] = c[i - 1, j]$ e $c[i, j] = c[i - 1, j - 1] + 1$, guardar $x[i]$ ($x[i]$ faz parte da LCS)
- Caso contrário, atualizamos $c[i, j]$ para a posição onde:
  - $c[i, j - 1] = c[i, j]$ **ou** $c[i - 1, j] = c[i, j]$
- Se $i = 0$ ou $j = 0$ (retorno chega ao fim)
- A saída é o conjunto de símbolos guardados em ordem inversa (pilha)

O Problema da Maior Subsequência Comum

- Fica assim:

i	j	0	1	2	3	4	5
		Yj	**B**	**D**	**C**	**A**	**B**
0	Xi	**0**	**0**	**0**	**0**	**0**	**0**
1	**A**	**0**	**0**	**0**	**0**	**1**	**1**
2	**B**	**0**	**1**	**1**	**1**	**1**	**2**
3	**C**	**0**	**1**	**1**	**2**	**2**	**2**
4	**B**	**0**	**1**	**1**	**2**	**2**	**3**

O Problema da Maior Subsequência Comum

- LCS (ordem inversa): **BCB**

i	j	0	1	2	3	4	5
		Yj	**B**	**D**	**C**	**A**	**B**
0	Xi	**0**	**0**	**0**	**0**	**0**	**0**
1	**A**	**0**	**0**	**0**	**0**	**1**	**1**
2	**B**	**0**	**1** ←	**1**	**1**	**1**	**2**
3	**C**	**0**	**1**	**1**	**2** ←	**2**	**2**
4	**B**	**0**	**1**	**1**	**2**	**2**	**3**

Dúvidas?